Министерство науки и высшего образования Российской Федерации

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

“НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО”

Дисциплина: Распределенные системы хранения данных

Лабораторная работа №2

Вариант: 706

Выполнила:
Карандашева Анастасия Денисовна
Группа: P3332

Преподаватель:
Афанасьев Дмитрий Борисович

г. Санкт-Петербург

2026

Содержание

Задание 3
Этап 1. Резервное копирование 4
Расчёт объёма резервных копий 7
Модель роста 7
Хранимые backup 7
Суммарный объём 7
Итог 7
Анализ 7
Этап 2. Потеря основного узла 8
Этап 3. Повреждение файлов БД 15
Этап 4. Логическое повреждение данных 22
Выводы 32

Задание

Цель работы - настроить процедуру периодического резервного копирования базы данных, сконфигурированной в ходе выполнения лабораторной работы №2, а также разработать и отладить сценарии восстановления в случае сбоев.

Узел из предыдущей лабораторной работы используется в качестве основного. Новый узел используется в качестве резервного. Учётные данные для подключения к новому узлу выдаёт преподаватель. В сценариях восстановления необходимо использовать копию данных, полученную на первом этапе данной лабораторной работы.

Этап 1. Резервное копирование

- Настроить резервное копирование с основного узла на резервный следующим образом:

Периодические полные копии с помощью SQL Dump.
По расписанию (cron) раз в сутки, методом SQL Dump с сжатием. Созданные архивы должны сразу перемещаться на резервный хост, они не должны храниться на основной системе. Срок хранения архивов на резервной системе - 4 недели. По истечении срока хранения, старые архивы должны автоматически уничтожаться.

- Подсчитать, каков будет объем резервных копий спустя месяц работы системы, исходя из следующих условий:
    - Средний объем новых данных в БД за сутки: 450МБ.
    - Средний объем измененных данных за сутки: 500МБ.
- Проанализировать результаты.

Этап 2. Потеря основного узла

Этот сценарий подразумевает полную недоступность основного узла. Необходимо восстановить работу СУБД на РЕЗЕРВНОМ узле, продемонстрировать успешный запуск СУБД и доступность данных.

Этап 3. Повреждение файлов БД

Этот сценарий подразумевает потерю данных (например, в результате сбоя диска или файловой системы) при сохранении доступности основного узла. Необходимо выполнить полное восстановление данных из резервной копии и перезапустить СУБД на ОСНОВНОМ узле.

Ход работы:

- Симулировать сбой:
    - удалить с диска директорию конфигурационных файлов СУБД со всем содержимым.
- Проверить работу СУБД, доступность данных, перезапустить СУБД, проанализировать результаты.

- Выполнить восстановление данных из резервной копии, учитывая следующее условие:
    - исходное расположение дополнительных табличных пространств недоступно - разместить в другой директории и скорректировать конфигурацию.
- Запустить СУБД, проверить работу и доступность данных, проанализировать результаты.

Этап 4. Логическое повреждение данных

Этот сценарий подразумевает частичную потерю данных (в результате нежелательной или ошибочной операции) при сохранении доступности основного узла. Необходимо выполнить восстановление данных на ОСНОВНОМ узле следующим способом:

- Восстановление с использованием архивных WAL файлов. (СУБД должна работать в режиме архивирования WAL, потребуется задать параметры восстановления).

Ход работы:

- В каждую таблицу базы добавить 2-3 новые строки, зафиксировать результат.
- Зафиксировать время и симулировать ошибку:
    - удалить каждую вторую строку в любой таблице (DELETE)
- Продемонстрировать результат.
- Выполнить восстановление данных указанным способом.
- Продемонстрировать и проанализировать результат.

Этап 1. Резервное копирование

Выполним подключение к резервному узлу и создадим директорию для бэкапов:

```
ssh -J s368273@helios.cs.ifmo.ru:2222 postgres2@pg199
mkdir backups
```

Вывод содержимого корневого каталога:

```
[postgres2@pg199 ~]$ ls -la
total 10
drwxr-xr-x   4 postgres2 postgres  4 26 марта 07:41 .
drwxr-xr-x  23 root      wheel    25 28 авг.   2024 ..
drwx------   2 postgres2 postgres  3  4 марта 19:38 .ssh
drwxr-xr-x   2 postgres2 postgres  2 26 марта 07:41 backups
```

Для настройки бэкапов подготовим скрипт pg_backup.bash, поместим на узел pg196 и запустим:

```
scp -J s368273@helios.cs.ifmo.ru:2222 *.bash postgres2@pg196:~/
```

```
[postgres2@pg196 ~]$ ./pg_backup.bash
```

```
pg_full_20260326_081835.sql.gz
```

Текст скрипта:

```
#!/usr/local/bin/bash

DB_HOST="pg196"
DB_USER="postgres2"
DB_PASSWORD="6lVL8X5a"
BACKUP_HOST="pg199"
BACKUP_USER="postgres2"
BACKUP_PATH="backups"
RETENTION_DAYS=28

export PGPASSWORD="$DB_PASSWORD"


# Используется текущая дата и время для избежания перезаписи старых бэкапов
BACKUP_NAME="pg_full_$(date +%Y%m%d_%H%M%S).sql.gz"
TEMP_BACKUP="/tmp/$BACKUP_NAME"


# pg_dump создаёт текстовый SQL-дамп (-F p), который через pipe передаётся в gzip
pg_dump -h "$DB_HOST" -U "$DB_USER" -d postgres -F p | gzip >
"$TEMP_BACKUP"

scp -o ProxyJump=s368273@helios.cs.ifmo.ru:2222 "$TEMP_BACKUP"
"$BACKUP_USER@$BACKUP_HOST:$BACKUP_PATH/"


# Удаление временного файла бэкапа из /tmp, очистка пароля
rm -f "$TEMP_BACKUP"
unset PGPASSWORD


# Через SSH с прыжком находим все файлы pg_full_*.sql.gz старше RETENTION_DAYS дней и
удаляем их
ssh -J s368273@helios.cs.ifmo.ru:2222 "$BACKUP_USER@$BACKUP_HOST" "find
$BACKUP_PATH -name 'pg_full_*.sql.gz' -mtime +$RETENTION_DAYS -delete"
```

Настроим запуск бэкапа на кроне: `crontab -e`

После добавления соответствующей строки:

```
[postgres2@pg196 ~]$ crontab -l
```

```
0 2 * * * /var/db/postgres2/pg1_backup.bash
```

Таким образом:

- Создан скрипт резервного копирования на основном узле pg196;
- Настроено ежедневное выполнение по расписанию через cron;
- Реализован SQL Dump с сжатием (pg_dump + gzip);
- Настроено перемещение архивов на резервный хост (scp на pg199);
- Архивы не хранятся на основной системе (удаляются из /tmp после передачи);
- Настроено автоматическое удаление старых архивов через find с параметром -mtime +28.

Запись с крона исчезает каждую ночь, поэтому для быстрого теста запись в кроне была установлена на ежеминутное выполнение скрипта бэкапа с логом:

```
[postgres2@pg196 ~]$ crontab -l

* * * * * /var/db/postgres2/pg1_backup >> /tmp/cron_debug.log 2>&1
```

В самом скрипте изменены следующие строки для удаления бэкапов, созданных более 5 минут назад:

```
RETENTION_MINUTES=5
ssh "$BACKUP_USER@$BACKUP_HOST" "find $BACKUP_PATH -name
'pg_full_*.sql.gz' -mmin +$RETENTION_MINUTES -delete"
```

Видно, что создание бэкапа работает корректно, новые версии ежеминутно появляются на узле, старые удаляются:

```
[postgres2@pg199 ~/backups]$ ls -la

total 44

drwxr-xr-x  2 postgres2 postgres    8 12 апр.  23:13 .

drwxr-xr-x  8 postgres2 postgres   12 12 апр.  22:19 ..

-rw-r--r--  1 postgres2 postgres 1462 12 апр.  23:08
pg_full_20260412_230800.sql.gz

-rw-r--r--  1 postgres2 postgres 1462 12 апр.  23:09
pg_full_20260412_230900.sql.gz

-rw-r--r--  1 postgres2 postgres 1462 12 апр.  23:10
pg_full_20260412_231000.sql.gz

-rw-r--r--  1 postgres2 postgres 1462 12 апр.  23:11
pg_full_20260412_231100.sql.gz

-rw-r--r--  1 postgres2 postgres 1462 12 апр.  23:12
pg_full_20260412_231200.sql.gz

-rw-r--r--  1 postgres2 postgres 1462 12 апр.  23:13
pg_full_20260412_231300.sql.gz
```

После проверки возвращены настройки по заданию.

Расчёт объёма резервных копий

Исходные данные:

- Новые данные за сутки: 450 МБ
- Измененные данные за сутки: 500 МБ
- Общий прирост данных в БД за сутки: 950 МБ
- Срок хранения: 28 дней
- Метод бэкапа: ежедневный полный SQL Dump
- начальный размер: ≈ 0.004 МБ (4.5К)
- коэффициент 3:1 (исходя из уровня сжатия gzip по умолчанию)

В расчётах примем начальный размер за 0, так как суточный рост значительно его превышает.

Модель роста

Размер базы на день t:
B(t) = 450 * t (МБ)
Размер сжатого backup:
D(t) = (450 * t) / 3 = 150 * t (МБ)

Хранимые backup

На 30-й день хранятся backup с 3 по 30 день (28 штук)

Суммарный объём

Total = сумма D(t), t = 3..30
Total = 150 * (3 + 4 + ... + 30)
Сумма:
S(1..30) = 30 * 31 / 2 = 465
S(1..2) = 3
S(3..30) = 465 - 3 = 462

Итог

Total = 150 * 462 = 69300 МБ ≈ 69.3 ГБ

Анализ

Дамп на 30-й день: 4.5 ГБ
Средний размер дампа: ≈ 2.4 ГБ/день
Пиковый объём: 4.5 ГБ (последний день)

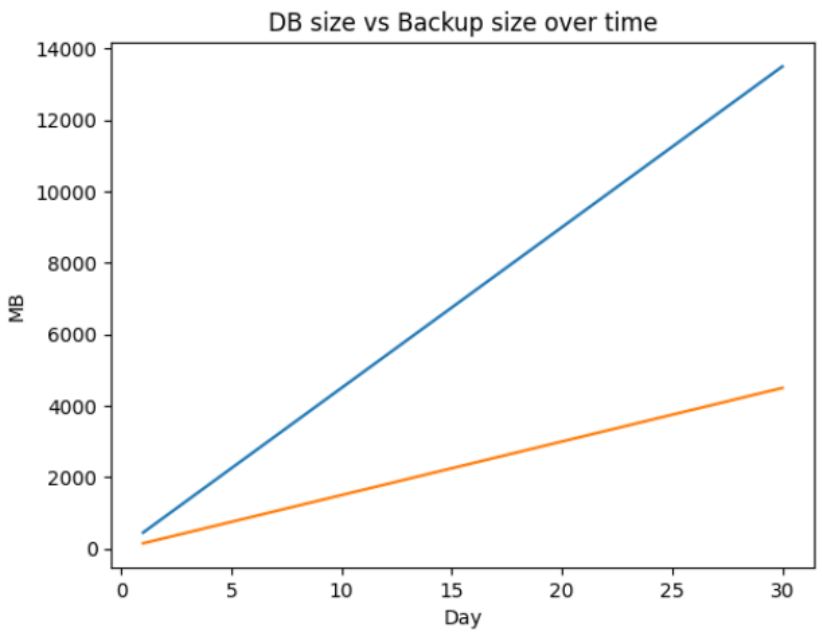

Рисунок 1. График роста размера базы данных и её бэкапа

Этап 2. Потеря основного узла

Полную недоступность основного узла будем имитировать остановкой PostgreSQL:

```
[postgres2@pg196 ~]$ pg_ctl -D $PGDATA -l $PGDATA/log/postgresql.log stop
ожидание завершения работы сервера.... готово
сервер остановлен
```

На начало этапа убедимся, что на резервном узле присутствует бэкап:

```
[postgres2@pg199 ~/backups]$ ls -la
total 18
drwxr-xr-x  2 postgres2 postgres    4 12 апр.  20:02 .
drwxr-xr-x  8 postgres2 postgres   12 12 апр.  20:05 ..
-rw-r--r--  1 postgres2 postgres 1438 28 марта 23:56
pg_full_20260328_235603.sql.gz
-rw-r--r--  1 postgres2 postgres 1488 12 апр.  17:20
pg_full_20260412_171914.sql.gz
```

Следующим шагом остановим сервер и очистим директорию $PGDATA, если ранее уже работали с PostgreSQL на узле, проинициализируем кластер и сконфигурируем сервер базы данных, опираясь на конфигурацию, ранее установленную на основном узле. Также заранее определим табличные пространства и роль для работы с пользовательской базой. Восстановим базу из последнего бэкапа в директории backups, перезапустим сервер базы данных, подключимся для проверки результата восстановления.

```
#!/usr/local/bin/bash

export PGDATA=/var/db/postgres2/data
```

```
export PGUSER=postgres2
export PGDATABASE=postgres
export BACKUP_DIR=~/backups
export PGPORT=9746

echo "[1/9] Остановка PostgreSQL..."
pg_ctl -D $PGDATA stop 2>/dev/null

echo "[2/9] Очистка старых данных..."
rm -rf $PGDATA/*

echo "[3/9] Создание директорий для табличных пространств..."
mkdir -p /var/db/postgres2/ts_new1
mkdir -p /var/db/postgres2/ts_new2

echo "[4/9] Инициализация новой БД (кодировка KOI8-R)..."
initdb -D $PGDATA --waldir=$HOME/wal --encoding=KOI8-R --
locale=ru_RU.KOI8-R

echo "[5/9] Настройка конфигурационных файлов..."
cat > $PGDATA/postgresql.conf << 'EOF'
port = 9746
listen_addresses = 'localhost'
max_connections = 7

shared_buffers = 2GB
temp_buffers = 64MB
work_mem = 128MB
effective_cache_size = 5GB

fsync = on
commit_delay = 100000
checkpoint_timeout = 30min

log_destination = 'stderr'
logging_collector = on
log_directory = 'log'
log_filename = 'postgresql.log'
log_min_messages = ERROR
log_disconnections = on
log_duration = on
EOF

cat > $PGDATA/pg_hba.conf << 'EOF'
# TYPE  DATABASE        USER            ADDRESS                 METHOD
local   loudwhiteuser   testuser                               peer
map=peermap
```

```
local   all             all                                     peer
map=peer2map
host    all             all             127.0.0.1/32            ident
map=peer2map
host    all             all             ::1/128                 ident
map=peer2map
host    all             all             0.0.0.0/0               reject
host    all             all             ::/0                    reject
EOF

cat >> $PGDATA/pg_ident.conf << 'EOF'
# Системный пользователь postgres2 может подключаться как testuser
peermap    postgres2    testuser
peer2map    postgres2    postgres2
EOF

mkdir -p $PGDATA/log

echo "[6/9] Запуск PostgreSQL..."
pg_ctl -D $PGDATA -l $PGDATA/log/postgresql.log start
sleep 3

echo "[7/9] Создание табличных пространств..."
psql -p $PGPORT -U $PGUSER -d postgres << EOF
CREATE TABLESPACE kls10 LOCATION '/var/db/postgres2/ts_new1';
CREATE TABLESPACE opy85 LOCATION '/var/db/postgres2/ts_new2';
CREATE ROLE testuser WITH LOGIN PASSWORD 'testpass';
GRANT CONNECT ON DATABASE template1 TO testuser;
CREATE DATABASE loudwhiteuser OWNER testuser TEMPLATE template1
ENCODING 'KOI8-R' LC_COLLATE 'ru_RU.KOI8-R' LC_CTYPE 'ru_RU.KOI8-R';
EOF

echo "[8/9] Восстановление данных из бэкапа..."
LATEST_BACKUP=$(ls -t $BACKUP_DIR/pg_full_*.sql.gz | head -1)
echo "      Используется бэкап: $(basename $LATEST_BACKUP)"

zcat $LATEST_BACKUP |  psql -p $PGPORT -U $PGUSER -d loudwhiteuser
UNPACKED="${LATEST_BACKUP%.gz}"

echo "[9/9] Перезапуск PostgreSQL и проверка..."
pg_ctl -D $PGDATA restart
sleep 2

psql -p $PGPORT -U $PGUSER -d postgres -c "SELECT version();"
psql -p $PGPORT -U $PGUSER -d postgres -c "\l"
psql -p $PGPORT -U $PGUSER -d postgres -c "\db"
psql -p 9746 -U testuser -d loudwhiteuser -c "\dn"
psql -p 9746 -U testuser -d loudwhiteuser -c "\dt"
```

```
psql -p 9746 -U testuser -d loudwhiteuser -c "\d"
```

Результат исполнения скрипта:

```
[1/9] Остановка PostgreSQL...
[2/9] Очистка старых данных...
[3/9] Создание директорий для табличных пространств...
[4/9] Инициализация новой БД (кодировка KOI8-R)...
Файлы, относящиеся к этой СУБД, будут принадлежать пользователю
"postgres2".
От его имени также будет запускаться процесс сервера.

Кластер баз данных будет инициализирован с локалью "ru_RU.KOI8-R".
Выбрана конфигурация текстового поиска по умолчанию "russian".

Контроль целостности страниц данных отключён.

создание каталога /var/db/postgres2/data... ок
создание каталога /var/db/postgres2/wal... ок
создание подкаталогов... ок
выбирается реализация динамической разделяемой памяти... posix
выбирается значение max_connections по умолчанию... 100
выбирается значение shared_buffers по умолчанию... 128MB
выбирается часовой пояс по умолчанию... Europe/Moscow
создание конфигурационных файлов... ок
выполняется подготовительный скрипт... ок
выполняется заключительная инициализация... ок
сохранение данных на диске... ок

initdb: предупреждение: включение метода аутентификации "trust" для
локальных подключений
initdb: подсказка: Другой метод можно выбрать, отредактировав pg_hba.conf
или ещё раз запустив initdb с ключом -A, --auth-local или --auth-host.

Готово. Теперь вы можете запустить сервер баз данных:

    pg_ctl -D /var/db/postgres2/data -l файл_журнала start

[5/9] Настройка конфигурационных файлов...
[6/9] Запуск PostgreSQL...
ожидание запуска сервера.... готово
сервер запущен
[7/9] Создание табличных пространств...
CREATE TABLESPACE
CREATE TABLESPACE
CREATE ROLE
GRANT
```

```
CREATE DATABASE
[8/9] Восстановление данных из бэкапа...
      Используется бэкап: pg_full_20260412_171914.sql.gz
SET
SET
SET
SET
SET
 set_config
------------

(1 строка)

SET
SET
SET
SET
SET
SET
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
SET
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
SET
CREATE TABLE
ALTER TABLE
SET
CREATE TABLE
ALTER TABLE
SET
CREATE TABLE
ALTER TABLE
SET
CREATE TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
```

```
COPY 8
COPY 6
COPY 3
COPY 3
COPY 3
 setval
--------
     38
(1 строка)

 setval
--------
     45
(1 строка)

SET
ALTER TABLE
SET
CREATE INDEX
SET
CREATE INDEX
SET
CREATE INDEX
SET
CREATE INDEX
SET
CREATE INDEX
ALTER INDEX
ALTER INDEX
ALTER INDEX
ALTER INDEX
[9/9] Перезапуск PostgreSQL и проверка...
ожидание завершения работы сервера.... готово
сервер остановлен
ожидание запуска сервера....2026-04-12 17:02:04.874 GMT [65639]
СООБЩЕНИЕ:  передача вывода в протокол процессу сбора протоколов
2026-04-12 17:02:04.874 GMT [65639] ПОДСКАЗКА:  В дальнейшем протоколы
будут выводиться в каталог "log".
 готово
сервер запущен

version
-----------------------------------------------------------------------------
-----------------------------------------------------------------------------
------------------------
 PostgreSQL 16.4 on amd64-portbld-freebsd14.1, compiled by FreeBSD clang
version 18.1.6 (https://github.com/llvm/llvm-project.git llvmorg-18.1.6-
0-g1118c2e05e67), 64-bit
```

```
(1 строка)

                                                                  Список баз
данных
      Имя       | Владелец  | Кодировка | Провайдер локали |  LC_COLLATE
|   LC_CTYPE    | локаль ICU | Правила ICU |      Права доступа
---------------+-----------+-----------+-----------------+-------------
+-------------+-----------+-----------+-------------+-----------------------
 loudwhiteuser | testuser  | KOI8R     | libc            | ru_RU.KOI8-R
| ru_RU.KOI8-R |           |           |
 postgres      | postgres2 | KOI8R     | libc            | ru_RU.KOI8-R
| ru_RU.KOI8-R |           |           |
 template0     | postgres2 | KOI8R     | libc            | ru_RU.KOI8-R
| ru_RU.KOI8-R |           |           | =c/postgres2          +
               |           |           |                 |
|              |           |           | postgres2=CTc/postgres2
 template1     | postgres2 | KOI8R     | libc            | ru_RU.KOI8-R
| ru_RU.KOI8-R |           |           | =c/postgres2          +
               |           |           |                 |
|              |           |           | postgres2=CTc/postgres2+
               |           |           |                 |
|              |           |           | testuser=c/postgres2
(4 строки)

            Список табличных пространств
    Имя     | Владелец  |       Расположение
------------+-----------+---------------------------
 kls10      | postgres2 | /var/db/postgres2/ts_new1
 opy85      | postgres2 | /var/db/postgres2/ts_new2
 pg_default | postgres2 |
 pg_global  | postgres2 |
(4 строки)

        Список схем
  Имя   |     Владелец
--------+------------------
 public | pg_database_owner
(1 строка)

                       Список отношений
 Схема  |      Имя      |           Тип            | Владелец
--------+---------------+--------------------------+----------
 public | products      | таблица                  | testuser
 public | sales         | секционированная таблица | testuser
 public | sales_2024_q1 | таблица                  | testuser
 public | sales_2024_q2 | таблица                  | testuser
 public | sales_2024_q3 | таблица                  | testuser
 public | sales_2024_q4 | таблица                  | testuser
```

```
(6 строк)

                        Список отношений
 Схема  |       Имя       |           Тип            | Владелец
--------+-----------------+--------------------------+----------
 public | products        | таблица                  | testuser
 public | products_id_seq | последовательность       | testuser
 public | sales           | секционированная таблица | testuser
 public | sales_2024_q1   | таблица                  | testuser
 public | sales_2024_q2   | таблица                  | testuser
 public | sales_2024_q3   | таблица                  | testuser
 public | sales_2024_q4   | таблица                  | testuser
 public | sales_id_seq    | последовательность       | testuser
(8 строк)
```

Таким образом, восстановление бд из бэкапа при полной потере доступа к основному узлу прошло успешно.

Этап 3. Повреждение файлов БД

Директорией конфигурационных файлов на узле pg196 является `$PGDATA`, то есть `$HOME/zfb92.` Для имитации повреждения конфигурационных файлов удалим содержимое директории.

```
#!/usr/local/bin/bash

pg_ctl -D $HOME/zfb92 stop 2>/dev/null

rm -rf $HOME/zfb92
```

Получаем вывод с информацией об остановке сервера:

```
[postgres2@pg196 ~]$ ./pg3_failure.bash
ожидание завершения работы сервера.... готово
сервер остановлен
```

Убедимся, что получили нужный результат, проверим работу СУБД и доступность данных.

```
#!/usr/local/bin/bash

psql -p 9746 -U testuser -d loudwhiteuser -c "SELECT now();"
echo "exit code: $?"
```

Видим, что, как и ожидалось, работа сервера нарушена:

```
[postgres2@pg196 ~]$ ./pg3_check_failure.bash

psql: ошибка: подключиться к серверу через сокет "/tmp/.s.PGSQL.9746" не
удалось: No such file or directory
```

```
        Сервер действительно работает локально и принимает подключения
через этот сокет?

exit code: 2
```

Заново проинициализируем кластер (так как все файлы конфигурации были удалены), определим содержимое конфигурационных файлов и выполним восстановление из бэкапа.

```
#!/usr/local/bin/bash

export PGDATA=$HOME/zfb92
export NEW_KLS=/var/db/postgres2/new_kls10
export NEW_OPY=/var/db/postgres2/new_opy85

mkdir -p $PGDATA
mkdir -p $NEW_KLS
mkdir -p $NEW_OPY

initdb -D $PGDATA --encoding=KOI8-R --locale=ru_RU.KOI8-R

mkdir -p $PGDATA/log

cat > $PGDATA/pg_hba.conf << 'EOF'
# TYPE  DATABASE        USER            ADDRESS                 METHOD
local   loudwhiteuser   testuser                               peer
map=peermap
local   all             all                                     peer
host    all             all             127.0.0.1/32            ident
host    all             all             ::1/128                 ident
host    all             all             0.0.0.0/0               reject
host    all             all             ::/0                    reject
EOF

cat >> $PGDATA/pg_ident.conf << 'EOF'
peermap    postgres2    testuser
EOF

cat > $PGDATA/postgresql.conf << 'EOF'
port = 9746
listen_addresses = 'localhost'
max_connections = 7

shared_buffers = 2GB
temp_buffers = 64MB
work_mem = 128MB
effective_cache_size = 5GB

fsync = on
```

```
commit_delay = 100000
checkpoint_timeout = 30min

log_destination = 'stderr'
logging_collector = on
log_directory = 'log'
log_filename = 'postgresql.log'
log_min_messages = ERROR
log_disconnections = on
log_duration = on
EOF

pg_ctl -D $PGDATA -l $PGDATA/log/postgresql.log start

sleep 5

psql -p 9746 -U postgres2 -d postgres << EOF
CREATE TABLESPACE kls10 LOCATION '$NEW_KLS';
CREATE TABLESPACE opy85 LOCATION '$NEW_OPY';
CREATE ROLE testuser WITH LOGIN PASSWORD 'testpass';
GRANT CONNECT ON DATABASE template1 TO testuser;
CREATE DATABASE loudwhiteuser OWNER testuser TEMPLATE template1
ENCODING 'KOI8-R' LC_COLLATE 'ru_RU.KOI8-R' LC_CTYPE 'ru_RU.KOI8-R';
GRANT CREATE ON TABLESPACE kls10 TO testuser;
GRANT CREATE ON TABLESPACE opy85 TO testuser;
EOF
```

Пока что стандартный вывод для инициализации кластера, настройки конфигурации и запуска сервера базы данных:

```
Файлы, относящиеся к этой СУБД, будут принадлежать пользователю
"postgres2".
От его имени также будет запускаться процесс сервера.

Кластер баз данных будет инициализирован с локалью "ru_RU.KOI8-R".
Выбрана конфигурация текстового поиска по умолчанию "russian".

Контроль целостности страниц данных отключён.

исправление прав для существующего каталога /var/db/postgres2/zfb92... ок
исправление прав для существующего каталога /var/db/postgres2/xda54... ок
создание подкаталогов... ок
выбирается реализация динамической разделяемой памяти... posix
выбирается значение max_connections по умолчанию... 100
выбирается значение shared_buffers по умолчанию... 128MB
выбирается часовой пояс по умолчанию... Europe/Moscow
создание конфигурационных файлов... ок
выполняется подготовительный скрипт... ок
```

```
выполняется заключительная инициализация... ок
сохранение данных на диске... ок

initdb: предупреждение: включение метода аутентификации "trust" для
локальных подключений
initdb: подсказка: Другой метод можно выбрать, отредактировав pg_hba.conf
или ещё раз запустив initdb с ключом -A, --auth-local или --auth-host.

Готово. Теперь вы можете запустить сервер баз данных:

    pg_ctl -D /var/db/postgres2/zfb92 -l файл_журнала start

ожидание запуска сервера.... готово
сервер запущен
CREATE TABLESPACE
CREATE TABLESPACE
CREATE ROLE
GRANT
CREATE DATABASE
GRANT
GRANT
```

```
#!/usr/local/bin/bash

export BACKUP_HOST=pg199
export BACKUP_USER=postgres2
export REMOTE_BACKUP_DIR=~/backups
export LOCAL_BACKUP_DIR=~/restore_tmp

mkdir -p $LOCAL_BACKUP_DIR

LATEST_BACKUP=$(ssh $BACKUP_USER@$BACKUP_HOST "ls -t
$REMOTE_BACKUP_DIR/pg_full_*.sql.gz | head -1")

scp $BACKUP_USER@$BACKUP_HOST:$LATEST_BACKUP $LOCAL_BACKUP_DIR/

LOCAL_FILE=$(ls -t $LOCAL_BACKUP_DIR/pg_full_*.sql.gz | head -1)

zcat $LOCAL_FILE | psql -p 9746 -U testuser -d loudwhiteuser
```

Результаты выполнения показывают, что данные действительно были восстановлены (правда, ещё стоит проверить, какие):

```
(postgres2@pg199.cs.ifmo.ru) Password for postgres2@pg199.cs.ifmo.ru:
(postgres2@pg199.cs.ifmo.ru) Password for postgres2@pg199.cs.ifmo.ru:
pg_full_20260328_235603.sql.gz
100% 1438   110.4KB/s   00:00
SET
```

```
SET
SET
SET
SET
 set_config
------------

(1 строка)

SET
SET
SET
SET
SET
SET
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
SET
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
ALTER SEQUENCE
SET
CREATE TABLE
ALTER TABLE
SET
CREATE TABLE
ALTER TABLE
SET
CREATE TABLE
ALTER TABLE
SET
CREATE TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
COPY 5
COPY 3
COPY 3
COPY 3
```

```
COPY 3
 setval
--------
      5
(1 строка)

 setval
--------
     12
(1 строка)

SET
ALTER TABLE
SET
CREATE INDEX
SET
CREATE INDEX
SET
CREATE INDEX
SET
CREATE INDEX
SET
CREATE INDEX
ALTER INDEX
ALTER INDEX
ALTER INDEX
ALTER INDEX
```

Последним этапом проверим результат восстановления, работу СУБД.

```
#!/usr/local/bin/bash

pg_ctl -D $HOME/zfb92 restart

sleep 2

psql -p 9746 -U testuser -d loudwhiteuser -c "SELECT version();"
psql -p 9746 -U testuser -d loudwhiteuser -c "\l"
psql -p 9746 -U testuser -d loudwhiteuser -c "\dt"
psql -p 9746 -U testuser -d loudwhiteuser -c "SELECT spcname FROM
pg_tablespace;"
```

Вывод в консоль:

```
ожидание завершения работы сервера..... готово
сервер остановлен
ожидание запуска сервера....2026-03-29 05:31:57.875 GMT [71794]
СООБЩЕНИЕ:  передача вывода в протокол процессу сбора протоколов
```

```
2026-03-29 05:31:57.875 GMT [71794] ПОДСКАЗКА:  В дальнейшем протоколы
будут выводиться в каталог "log".
 готово
сервер запущен

version
-------------------------------------------------------------------------
-------------------------------------------------------------------------
------------------------
 PostgreSQL 16.4 on amd64-portbld-freebsd14.1, compiled by FreeBSD clang
version 18.1.6 (https://github.com/llvm/llvm-project.git llvmorg-18.1.6-
0-g1118c2e05e67), 64-bit
(1 строка)

                                                              Список баз
данных
      Имя      | Владелец  | Кодировка | Провайдер локали |  LC_COLLATE
|   LC_CTYPE   | локаль ICU | Правила ICU |      Права доступа
---------------+-----------+-----------+------------------+-------------
+-------------+------------+------------+-------------------------
 loudwhiteuser | testuser  | KOI8R     | libc             | ru_RU.KOI8-R
| ru_RU.KOI8-R |            |            |
 postgres      | postgres2 | KOI8R     | libc             | ru_RU.KOI8-R
| ru_RU.KOI8-R |            |            |
 template0     | postgres2 | KOI8R     | libc             | ru_RU.KOI8-R
| ru_RU.KOI8-R |            |            | =c/postgres2          +
               |           |           |                  |
|              |            |            | postgres2=CTc/postgres2
 template1     | postgres2 | KOI8R     | libc             | ru_RU.KOI8-R
| ru_RU.KOI8-R |            |            | =c/postgres2          +
               |           |           |                  |
|              |            |            | postgres2=CTc/postgres2+
               |           |           |                  |
|              |            |            | testuser=c/postgres2
(4 строки)

                       Список отношений
 Схема  |      Имя      |           Тип            | Владелец
--------+---------------+--------------------------+----------
 public | products      | таблица                  | testuser
 public | sales         | секционированная таблица | testuser
 public | sales_2024_q1 | таблица                  | testuser
 public | sales_2024_q2 | таблица                  | testuser
 public | sales_2024_q3 | таблица                  | testuser
 public | sales_2024_q4 | таблица                  | testuser
(6 строк)

  spcname
```

```
------------
 pg_default
 pg_global
 kls10
 opy85
(4 строки)
```

Для повреждения конфигурационных файлов базы данных сценарий восстановления также был успешно завершён.

Этап 4. Логическое повреждение данных

Подготовительный этап: так как на этапе 3 мы работали с той же директорией $PGDATA и меняли табличные пространства относительно исходных, очистим директорию и повторим процесс инициализации кластера, конфигурации сервера и заполнения базы данных (будем использовать этот скрипт при отладке для возврата к началу).

```
#!/usr/local/bin/bash

pg_ctl -D $HOME/zfb92 stop
rm -rf zfb92
rm -rf xda54
rm -rf kls10
rm -rf opy85
rm -rf wal_archive
rm -rf base_backup
```

```
#!/usr/local/bin/bash

mkdir -p $HOME/zfb92
initdb -D $HOME/zfb92 --waldir=$HOME/xda54 --encoding=KOI8-R --
locale=ru_RU.KOI8-R

export PGDATA=/var/db/postgres2/zfb92
export PGUSER=postgres2
export PGDATABASE=postgres
export PGPORT=9746

mkdir -p $HOME/zfb92/log

cat > $PGDATA/postgresql.conf << 'EOF'

port = 9746
listen_addresses = 'localhost'
```

```
max_connections = 7

shared_buffers = 2GB
temp_buffers = 64MB
work_mem = 128MB
effective_cache_size = 5GB

fsync = on
commit_delay = 100000
checkpoint_timeout = 30min

log_destination = 'stderr'
logging_collector = on
log_directory = 'log'
log_filename = 'postgresql.log'
log_min_messages = ERROR
log_disconnections = on
log_duration = on
EOF

cat > $PGDATA/pg_hba.conf << 'EOF'
# TYPE  DATABASE        USER            ADDRESS                 METHOD
local   loudwhiteuser   testuser                               peer
map=peermap
local   all             all                                     peer
map=peer2map
host    all             all             127.0.0.1/32            ident
map=peer2map
host    all             all             ::1/128                 ident
map=peer2map
host    all             all             0.0.0.0/0               reject
host    all             all             ::/0                    reject
EOF

cat >> $PGDATA/pg_ident.conf << 'EOF'
# Системный пользователь postgres2 может подключаться как testuser
peermap    postgres2    testuser
peer2map    postgres2    postgres2
EOF

mkdir -p /var/db/postgres2/kls10
mkdir -p /var/db/postgres2/opy85

pg_ctl -l $HOME/zfb92/log/postgresql.log start

psql -p $PGPORT -U $PGUSER -d postgres << EOF
CREATE TABLESPACE kls10 LOCATION '/var/db/postgres2/kls10';
CREATE TABLESPACE opy85 LOCATION '/var/db/postgres2/opy85';
```

```
CREATE ROLE testuser WITH LOGIN PASSWORD 'testpass';
GRANT CONNECT ON DATABASE template1 TO testuser;
CREATE DATABASE loudwhiteuser OWNER testuser TEMPLATE template1
ENCODING 'KOI8-R' LC_COLLATE 'ru_RU.KOI8-R' LC_CTYPE 'ru_RU.KOI8-R';
GRANT CREATE ON TABLESPACE kls10 TO testuser;
GRANT CREATE ON TABLESPACE opy85 TO testuser;
EOF

psql -p 9746 -U testuser -d loudwhiteuser << EOF
DROP TABLE IF EXISTS sales CASCADE;
CREATE TABLE sales (
    id SERIAL,
    sale_date DATE NOT NULL,
    amount DECIMAL(10,2),
    product_id INTEGER
) PARTITION BY RANGE (sale_date);

CREATE TABLE sales_2024_q1 PARTITION OF sales
    FOR VALUES FROM ('2024-01-01') TO ('2024-04-01')
    TABLESPACE kls10;

CREATE TABLE sales_2024_q2 PARTITION OF sales
    FOR VALUES FROM ('2024-04-01') TO ('2024-07-01')
    TABLESPACE opy85;

CREATE TABLE sales_2024_q3 PARTITION OF sales
    FOR VALUES FROM ('2024-07-01') TO ('2024-10-01')
    TABLESPACE kls10;

CREATE TABLE sales_2024_q4 PARTITION OF sales
    FOR VALUES FROM ('2024-10-01') TO ('2025-01-01')
    TABLESPACE opy85;

CREATE INDEX idx_sales_2024_q1_amount ON sales_2024_q1(amount)
TABLESPACE kls10;
CREATE INDEX idx_sales_2024_q2_amount ON sales_2024_q2(amount)
TABLESPACE opy85;
CREATE INDEX idx_sales_2024_q3_amount ON sales_2024_q3(amount)
TABLESPACE kls10;
CREATE INDEX idx_sales_2024_q4_amount ON sales_2024_q4(amount)
TABLESPACE opy85;

INSERT INTO sales (sale_date, amount, product_id) VALUES
    ('2024-01-15', 100.50, 1),
    ('2024-02-20', 200.75, 2),
    ('2024-03-10', 150.25, 1),
    ('2024-04-05', 300.00, 3),
    ('2024-05-12', 250.50, 2),
```

```
    ('2024-06-18', 175.80, 1),
    ('2024-07-22', 400.20, 4),
    ('2024-08-30', 350.00, 3),
    ('2024-09-14', 225.90, 2),
    ('2024-10-01', 500.00, 5),
    ('2024-11-11', 275.60, 1),
    ('2024-12-25', 600.00, 4);

CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    price DECIMAL(10,2)
) TABLESPACE kls10;

CREATE INDEX idx_sales_amount ON sales(amount) TABLESPACE opy85;

INSERT INTO products (name, price) VALUES
    ('Product A', 100.00),
    ('Product B', 200.00),
    ('Product C', 300.00),
    ('Product D', 400.00),
    ('Product E', 500.00);

SELECT
    spcname AS tablespace_name,
    pg_tablespace_location(oid) AS location
FROM pg_tablespace;

-- Список объектов по табличным пространствам
SELECT
    t.spcname AS tablespace_name,
    c.relname AS object_name,
    c.relkind AS object_type
FROM pg_class c
JOIN pg_tablespace t ON c.reltablespace = t.oid
WHERE c.reltablespace > 0
ORDER BY t.spcname, c.relname;

-- Информация о партициях
SELECT
    inhrelid::regclass AS partition_name,
    pg_get_expr(relpartbound, inhrelid) AS partition_bound,
    (SELECT spcname FROM pg_tablespace WHERE oid = reltablespace) AS
tablespace
FROM pg_inherits
JOIN pg_class ON inhrelid = pg_class.oid
WHERE inhparent = 'sales'::regclass;
```

```
SELECT
    'kls10' AS tablespace,
    relname AS object_name,
    relkind AS object_type
FROM pg_class
WHERE reltablespace = (SELECT oid FROM pg_tablespace WHERE spcname =
'kls10')
ORDER BY relname;

SELECT
    'opy85' AS tablespace,
    relname AS object_name,
    relkind AS object_type
FROM pg_class
WHERE reltablespace = (SELECT oid FROM pg_tablespace WHERE spcname =
'opy85')
ORDER BY relname;
EOF
```

Так как скрипт повторяет действия первой лабораторной работы, консольный вывод здесь не будем приводить повторно. Главное, что на текущий момент мы получаем работающий сервер с заполненным данными в пользовательской базе.

Следующий шаг — включить архивирование. Для этого меняем конфигурацию, добавляя соответствующие параметры.

```
#!/usr/local/bin/bash

psql -p 9746 -U postgres2 -d postgres -c "ALTER SYSTEM SET wal_level =
'replica';"
psql -p 9746 -U postgres2 -d postgres -c "ALTER SYSTEM SET archive_mode
= on;"
psql -p 9746 -U postgres2 -d postgres -c "ALTER SYSTEM SET
archive_command = 'test ! -f $HOME/wal_archive/%f && cp %p
$HOME/wal_archive/%f';"

mkdir -p ~/wal_archive

pg_ctl -D $HOME/zfb92 restart
psql -p 9746 -U postgres2 -d postgres -c "SHOW archive_mode; SHOW
archive_command;"
```

Видим, что архивирование включено:

```
ALTER SYSTEM
ALTER SYSTEM
ALTER SYSTEM
ожидание завершения работы сервера..... готово
сервер остановлен
```

```
ожидание запуска сервера....2026-04-12 18:35:33.291 GMT [74963]
СООБЩЕНИЕ:  передача вывода в протокол процессу сбора протоколов
2026-04-12 18:35:33.291 GMT [74963] ПОДСКАЗКА:  В дальнейшем протоколы
будут выводиться в каталог "log".
 готово
сервер запущен
 archive_mode
--------------
 on
(1 строка)

                                   archive_command
-----------------------------------------------------------------------
-------------
 test ! -f /var/db/postgres2/wal_archive/%f && cp %p
/var/db/postgres2/wal_archive/%f
(1 строка)
```

Для восстановления (PITR) нам понадобится базовый бэкап. Сохраним перед изменениями:

```
#!/usr/local/bin/bash

export PGDATA=$HOME/zfb92

cat >> $PGDATA/pg_hba.conf <<EOF
host    replication  postgres2  ::1/128       trust
host    replication  postgres2  127.0.0.1/32  trust
EOF

pg_ctl -D $PGDATA restart
pg_basebackup -D $HOME/base_backup -h localhost -p 9746 -U postgres2 -P
-Ft -Xs -z
```

```
ожидание завершения работы сервера.... готово
сервер остановлен
ожидание запуска сервера....2026-04-12 18:49:38.111 GMT [76397]
СООБЩЕНИЕ:  передача вывода в протокол процессу сбора протоколов
2026-04-12 18:49:38.111 GMT [76397] ПОДСКАЗКА:  В дальнейшем протоколы
будут выводиться в каталог "log".
 готово
сервер запущен
pg_basebackup: ошибка: подключиться к серверу через сокет
"/tmp/.s.PGSQL.9746" не удалось: ВАЖНО:  в pg_hba.conf нет записи,
разрешающей подключение для репликации с компьютера "[local]" для
пользователя "postgres2", без шифрования
[postgres2@pg196 ~]$ pg_ctl -D $PGDATA restart
pg_basebackup -D $HOME/base_backup -h localhost -p 9746 -U postgres2 -P -
Ft -Xs -z
```

```
ожидание завершения работы сервера.... готово
сервер остановлен
ожидание запуска сервера....2026-04-12 18:50:04.214 GMT [76635]
СООБЩЕНИЕ:  передача вывода в протокол процессу сбора протоколов
2026-04-12 18:50:04.214 GMT [76635] ПОДСКАЗКА:  В дальнейшем протоколы
будут выводиться в каталог "log".
 готово
сервер запущен
31349/31349 КБ (100%), табличное пространство 3/3
```

Видим, что все 3 табличных пространства были зафиксированы, бэкап успешно сохранён.

Далее по зданию добавим по несколько строк в каждую из пользовательских таблиц:

```
#!/usr/local/bin/bash

export PGUSER=testuser
export PGDATABASE=loudwhiteuser

psql -p 9746 -U $PGUSER -d $PGDATABASE <<EOF
INSERT INTO products (name, price) VALUES
('prodA', 10.5),
('prodB', 11.0),
('prodC', 12.0);

INSERT INTO sales (sale_date, amount, product_id) VALUES
('2024-01-15', 20.00, 1),
('2024-02-15', 35.50, 2),
('2024-03-15', 50.00, 3);

SELECT * FROM products ORDER BY id;
SELECT * FROM sales ORDER BY id;
EOF
```

```
INSERT 0 3
INSERT 0 3
 id |   name    | price
----+-----------+--------
  1 | Product A | 100.00
  2 | Product B | 200.00
  3 | Product C | 300.00
  4 | Product D | 400.00
  5 | Product E | 500.00
  6 | prodA     |  10.50
  7 | prodB     |  11.00
  8 | prodC     |  12.00
(8 строк)

 id | sale_date  | amount | product_id
```

```
----+------------+--------+------------
  1 | 2024-01-15 | 100.50 |          1
  2 | 2024-02-20 | 200.75 |          2
  3 | 2024-03-10 | 150.25 |          1
  4 | 2024-04-05 | 300.00 |          3
  5 | 2024-05-12 | 250.50 |          2
  6 | 2024-06-18 | 175.80 |          1
  7 | 2024-07-22 | 400.20 |          4
  8 | 2024-08-30 | 350.00 |          3
  9 | 2024-09-14 | 225.90 |          2
 10 | 2024-10-01 | 500.00 |          5
 11 | 2024-11-11 | 275.60 |          1
 12 | 2024-12-25 | 600.00 |          4
 13 | 2024-01-15 |  20.00 |          1
 14 | 2024-02-15 |  35.50 |          2
 15 | 2024-03-15 |  50.00 |          3
 (15 строк)
```

Будем симулировать ошибку. Для этого сначала зафиксируем время сбоя, а затем по заданию удалим каждую вторую строку:

```
SCHEMA_TIME=$(psql -h localhost -p 9746 -U postgres2 -d loudwhiteuser -
t -c "SELECT now();")
echo "Цель восстановления: $SCHEMA_TIME"

psql -h localhost -p 9746 -U testuser -d loudwhiteuser -c "DELETE FROM
products WHERE id % 2 = 1;"
```

```
psql -h localhost -p 9746 -U testuser -d loudwhiteuser -c "SELECT
count(*) FROM products; SELECT count(*) FROM sales;"
```

```
echo "Цель восстановления: $SCHEMA_TIME"
Цель восстановления:  2026-04-12 18:54:20.017887+00
DELETE 4
count
-------
    4
(1 строка)

 count
-------
   15
(1 строка)
```

Таким образом, из таблицы products была удалена половина элементов, стало 4 вместо 8. Таблица sales осталась неизменной.

Для восстановления остановим сервер бд и используем базовый бэкап. Добавим restore_command, ранее сохраненное время сбоя и создадим сигнальный файл для запуска в режиме восстановления.

```
#!/usr/local/bin/bash

pg_ctl -D $HOME/zfb92 stop
rm -rf $HOME/zfb92/*
tar -xvf $HOME/base_backup/base.tar.gz -C $HOME/zfb92/

cat >> $HOME/zfb92/postgresql.conf <<EOF
restore_command = 'cp $HOME/wal_archive/%f %p'
recovery_target_time = '$SCHEMA_TIME'
EOF

touch $HOME/zfb92/recovery.signal

pg_ctl -D $HOME/zfb92 -l $HOME/zfb92/recovery.log start
tail -f $HOME/zfb92/recovery.log

psql -p 9746 -U testuser -d loudwhiteuser <<EOF
SELECT * FROM products ORDER BY id;
SELECT * FROM sales ORDER BY id;
EOF
```

```
ожидание запуска сервера..... готово
сервер запущен
2026-04-12 19:07:36.086 GMT [78516] СООБЩЕНИЕ:  передача вывода в
протокол процессу сбора протоколов
2026-04-12 19:07:36.086 GMT [78516] ПОДСКАЗКА:  В дальнейшем протоколы
будут выводиться в каталог "log".
```

Консольный вывод также содержит список всех восстановленных данных, но очень уж длинный. По приведенной финальной части видно, что восстановление прошло успешно и сервер бд смог запуститься.

Далее можно увидеть, что данные таблицы products действительно восстановились до 8 строк:

```
SELECT * FROM products ORDER BY id;
SELECT * FROM sales ORDER BY id;
EOF
 id |   name    | price
----+-----------+--------
  1 | Product A | 100.00
  2 | Product B | 200.00
  3 | Product C | 300.00
  4 | Product D | 400.00
  5 | Product E | 500.00
```

```
  6 | prodA     |  10.50
  7 | prodB     |  11.00
  8 | prodC     |  12.00
(8 строк)

 id | sale_date  | amount | product_id
----+------------+--------+------------
  1 | 2024-01-15 | 100.50 |          1
  2 | 2024-02-20 | 200.75 |          2
  3 | 2024-03-10 | 150.25 |          1
  4 | 2024-04-05 | 300.00 |          3
  5 | 2024-05-12 | 250.50 |          2
  6 | 2024-06-18 | 175.80 |          1
  7 | 2024-07-22 | 400.20 |          4
  8 | 2024-08-30 | 350.00 |          3
  9 | 2024-09-14 | 225.90 |          2
 10 | 2024-10-01 | 500.00 |          5
 11 | 2024-11-11 | 275.60 |          1
 12 | 2024-12-25 | 600.00 |          4
 13 | 2024-01-15 |  20.00 |          1
 14 | 2024-02-15 |  35.50 |          2
 15 | 2024-03-15 |  50.00 |          3
(15 строк)
```

Таблица вернулась в состояние до удаления половины элементов. После этого не забудем прописать функцию pg_wal_replay_resume() чтобы завершить восстановление. После этого можно спокойно продолжать работать с БД. Если не прописать эту функцию, тогда у нас восстановление не завершится и при перезапуске без файла recovery.signal у нас все откатится к последней версии как было после DELETE.

```
#!/usr/local/bin/bash

psql -p 9746 -U postgres2 -d postgres -c "SELECT
pg_wal_replay_resume();"
rm $HOME/zfb92/recovery.signal

sed -i '' '/^restore_command/d' $HOME/zfb92/postgresql.conf
sed -i '' '/^recovery_target_time/d' $HOME/zfb92/postgresql.conf

# Проверка результатов вне режима восстановления:
psql -p 9746 -U postgres2 -d postgres -c "SELECT pg_is_in_recovery(),
pg_last_wal_replay_lsn(), pg_last_xact_replay_timestamp();"

# Проверка восстановленных данных
psql -p 9746 -U testuser -d loudwhiteuser -c "SELECT count(*) FROM
products;"
```

По выводу можно судить о том, что база данных больше не находиться в режиме восстановления (pg_is_in_recovery = f, то есть false):

```
[postgres2@pg196 ~]$ psql -p 9746 -U postgres2 -d postgres -c "SELECT
pg_is_in_recovery(), pg_last_wal_replay_lsn(),
pg_last_xact_replay_timestamp();"
 pg_is_in_recovery | pg_last_wal_replay_lsn |
pg_last_xact_replay_timestamp
-------------------+------------------------+-----------------------------
---
 f                 | 0/6000000              |
(1 строка)
```

Выводы

В ходе лабораторной работы настроено автоматическое ежедневное резервное копирование PostgreSQL с передачей SQL-дампов на резервный узел и хранением 28 дней. Успешно отработаны три сценария восстановления: полная потеря основного узла (восстановление из дампа на резервном узле), повреждение файлов БД (восстановление с переносом табличных пространств в новые директории) и логическое повреждение данных (PITR-восстановление с использованием WAL-архивов до момента времени перед ошибочным удалением). Все сценарии завершились успешно, данные полностью восстановлены, СУБД работоспособна.